

ECE 657D: Research Project

Is Edge AI the Future? Exploring On-Device Neural Net Inference on Microcontrollers.

Tilak Raval — 21208713

University of Waterloo — Winter 2026

Q1: Research Question

I come from a field of robotics and IoT automation, and having a particular interest in the underlying implementation of artificial intelligence (specifically, neural networks). The question I ask is "Can we run a neural network model on a tiny microcontroller (ESP32-S3) (1) and still obtain some useful and accurate results?" To test this, I will build a simple on-device image classifier for fruits/vegetables and measure: "how many classes it can handle reliably, how accurate it is in real conditions (i.e., lighting, noise), and how accuracy trades off against latency and limited memory and compute on the embedded hardware.

Q2: Prior Work and Limitations

According to my search on Microcontroller-based Edge AI applications, TinyML (2; 3) and toolchain guides (e.g., Edge Impulse and Seeed Studio XIAO ESP32S3 Sense docs). (1) (4) (5) show that quantized CNNs (6; 7) can run on microcontrollers with TensorFlow Lite Micro, confirming feasibility on ESP32-class devices; however, these examples typically fix the input resolution and compute per - frame and report end-to-end accuracy-latency tradeoffs or lighting robustness with live cameras. Research on cascaded/early-exit (8; 9) inference shows that confidence-based early stopping can reduce average computation with small accuracy loss, but evaluations mostly target larger models and non-embedded platforms, leaving open whether a simple two-stage, confidence-triggered policy is effective under tight MCU memory and camera I/O constraints. Work on dynamic-resolution networks further indicates that different inputs warrant different resolutions to save compute, yet this is rarely tested on microcontrollers with strict RAM/PSRAM and latency limits.

Q3: Novelty of Approach

This project brings adaptive inference to embedded devices, a camera-based microcontroller setting, on the XIAO ESP32S3 Sense. The novelty is a practical combination of techniques not jointly tested on this device: **Confidence-triggered compute scaling:** run a fast low-resolution pass first, then scale up resolution/processing only when confidence is low.

Light-weight pipeline: pair lightweight on-device pre-processing with training-time augmentation to handle real-world illumination changes.

Systematic on-device evaluation: measure accuracy, latency, and throughput on the actual microcontroller (not on CPU/cloud).

Q4: Impact and Motivation

Who benefits? Students, makers, and educators: a reproducible TinyML recipe with clear numbers for accuracy vs. latency on real hardware. IoT, robotics prototypes: fast, offline, privacy - preservation vision for

low-power devices (E.g., accessibility tools, retail counters (E.g., foodbasic self Checkout), and IoT nodes).

Why should we care? It shows how to push deep learning to the sensor for lower latency, better privacy, and lower energy—key requirements where cloud inference is impractical.

Q5: Risks and Assumptions

Main risks: The ESP32-S3’s limited RAM/compute may restrict model size or reduce accuracy across multiple classes. Lighting variation and camera noise may still hurt performance despite preprocessing. Adaptive inference may fail to improve latency if confidence scores are unstable.

Key assumptions: The quantized model fits in RAM/PSRAM, and real-time capture + inference stay within timing limits. Training-time augmentation and lightweight pre-processing will provide enough robustness for real-world testing.

Mitigation: Use a fixed-resolution baseline, conservative memory planning, and incremental testing to ensure the system works even if adaptive inference provides limited gains.

Q6: Resources

Compute: Training will be done on a CPU or Google Colab, which is sufficient for small TinyML models.

Deployment: All inference runs on the Seeed Studio XIAO ESP32S3 Sense, which has an onboard camera and supports microSD storage up to 32GB (FAT32) for models or data.

Data: Most data will come from device-captured images, ensuring realistic lighting and noise. A small external fruit/vegetable dataset may be used only for light pretraining or augmentation, with final testing done entirely on device-captured samples.

Q7: Weekly Plan

Week	Plan
1 (Mar 3–7)	Set up environment, collect sample, train baseline model, deploy on ESP32-S3.
2 (Mar 10–14)	Measure baseline accuracy/latency; refine preprocessing and add basic lighting augmentation.
3 (Mar 17–21)	Add lightweight on-device preprocessing and re-test robustness under varied lighting.
4 (Mar 24–28)	Implement confidence-based adaptive inference and compare against the baseline.
5 (Mar 31–Apr 2)	Final evaluation, generate plots/tables, and complete the report and code cleanup.

Q8: Milestones and Success Criteria

Success Measures: (1) Accuracy/F1: meets or exceeds the fixed-resolution baseline. (2) Latency/FPS: equal or lower average latency, stable real-time performance. (3) Robustness: handles basic lighting variation without major accuracy drop.

Baselines: (1) Fixed-Resolution Baseline: no preprocessing, no adaptive logic. (2) Lighting-Augmented Baseline: training-time augmentation only. (3) Adaptive Inference: confidence-triggered low→high-resolution scaling.

Success Condition Adaptive inference shows better robustness or lower latency without sacrificing accuracy.

References

- [1] S. Studio, “Xiao esp32s3 sense wiki,” <https://wiki.seeedstudio.com/xiao-esp32s3-sense/>, 2024.
- [2] G. Gigwegbe, “tinymml-papers-and-projects,” <https://github.com/gigwegbe/tinymml-papers-and-projects>, Dec. 2020, gitHub repository.
- [3] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, “A comprehensive survey on tinymml,” *IEEE Access*, vol. 11, pp. 96 892–96 922, 2023.
- [4] TensorFlow, “Tensorflow lite for microcontrollers,” <https://www.tensorflow.org/lite/microcontrollers>, 2024.
- [5] E. Impulse, “Edge impulse documentation,” <https://docs.edgeimpulse.com>, 2024.
- [6] T. Garbay, K. Hachicha, P. Dobias, W. Dron, P. Lusich, I. Khalis, A. Pinna, and B. Granado, “Accurate estimation of the cnn inference cost for tinymml devices,” in *2022 IEEE 35th International System-on-Chip Conference (SOCC)*, 2022, pp. 1–6.
- [7] A. S. Baslamisli, H.-A. Le, and T. Gevers, “Cnn based learning using reflection and retinex models for intrinsic image decomposition,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6674–6683.
- [8] J. Redmon and A. Farhadi, “Yolo9000: Better, faster, stronger,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6517–6525.
- [9] S. Teerapittayanon, B. McDanel, and H. T. Kung, “Branchynet: Fast inference via early exiting from deep neural networks,” in *International Conference on Pattern Recognition (ICPR)*, Dec. 2016, pp. 5308–5313, accessed: 2026-02-27.